

北京邮电大学

网络空间安全学院



《软件安全》第一次实验报告

姓 名 _____

学 号 _____

班 级 _____

邮 箱

2025 年 10 月 9 日

软件安全第一次实验报告

一、实验目标

1. 读懂实验所给的用于密码验证的小程序，改变程序的运行路线，系统：Windows 2000~Windows 11、wsl、Linux 皆可；工具：VS、gdb、Ollydbg，x96dbg 等。（以上记作实验一）

2. 根据在附件/test 目录下的源码；学习用调试工具调试密码验证小程序；通过修改汇编语句来修改程序的判断条件，改变程序的运行路线。（以上记作实验二）

3. 根据在附件/crackme 目录下的文件，修改程序代码的方式绕过 crackme.exe 的密码验证逻辑，至少采用 2 种破解方式方法，并提交破解后的程序。（以上记作实验三）

4. 报告要求

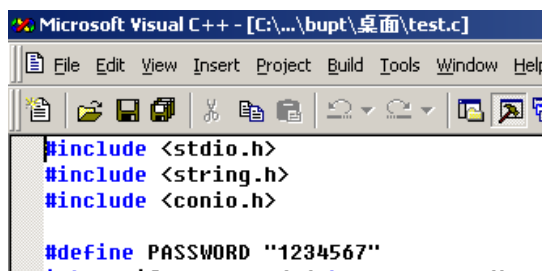
请详述修改过程；实验结果需要截图证明；绘出程序运行的结构图，以及修改原理的图示（通过程序运行的结构图）。

二、实验步骤与结果

（一）读懂实验所给的用于密码验证的小程序

使用 VC 打开 test.c，对密码验证小程序代码分析如下：

1. 程序定义了一个字符串常量 PASSWORD，其值为 1234567。



2. 程序定义了一个 verify_password() 函数，函数传入参数为一个字符数组，返回值为一个整型变量。

```
int verify_password (char *password)
{
    int authenticated;
    char buffer[8];
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password);

    return authenticated;
}
```

此函数所实现的功能为：将传入的字符串与 PASSWORD 比较，返回比较结果

authenticated。若 authenticated=0 则字符串不同，验证失败；若 authenticated=1 则字符串相同，验证成功。

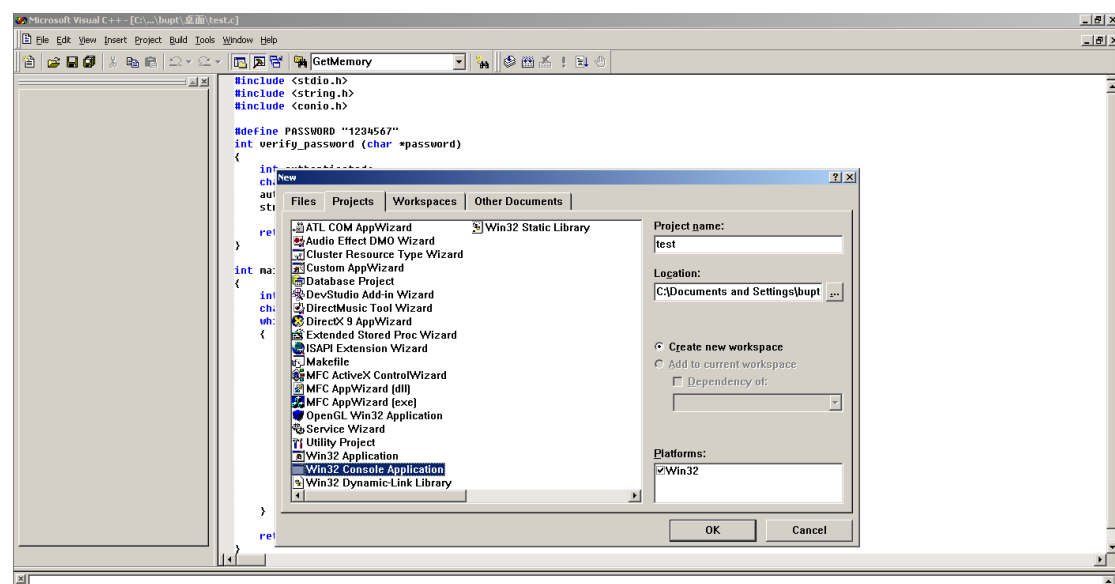
3.主函数中，由 while(1)控制不断循环，每次循环中首先输入一个字符串 password，再将 password 作为参数输入 verify_password 函数进行验证。若验证成功，则输出“Congratulation! You have passed the verification!\n”并跳出循环、结束程序；若验证失败，则输出“incorrect password!\n\n”，开始下一轮循环。

```
int main()
{
    int valid_flag=0;
    char password[1024];
    while(1)
    {
        printf("please input password:      ");
        scanf("%s",password);
        valid_flag = verify_password(password);
        if(valid_flag)
        {
            printf("incorrect password!\n\n");
        }
        else
        {
            printf("Congratulation! You have passed the verification!\n Press Enter to Exit!\n");
            getch();
            break;
        }
    }

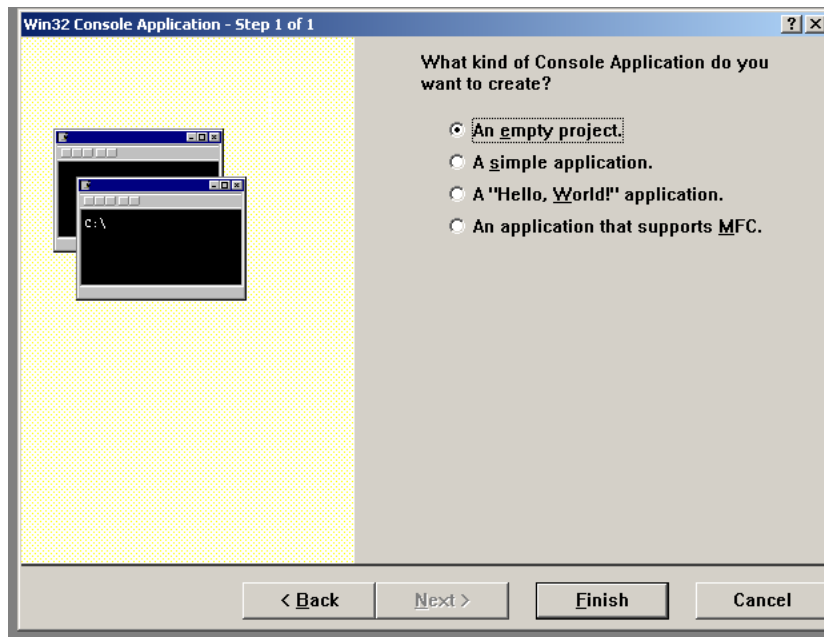
    return 0;
}
```

4.生成工程

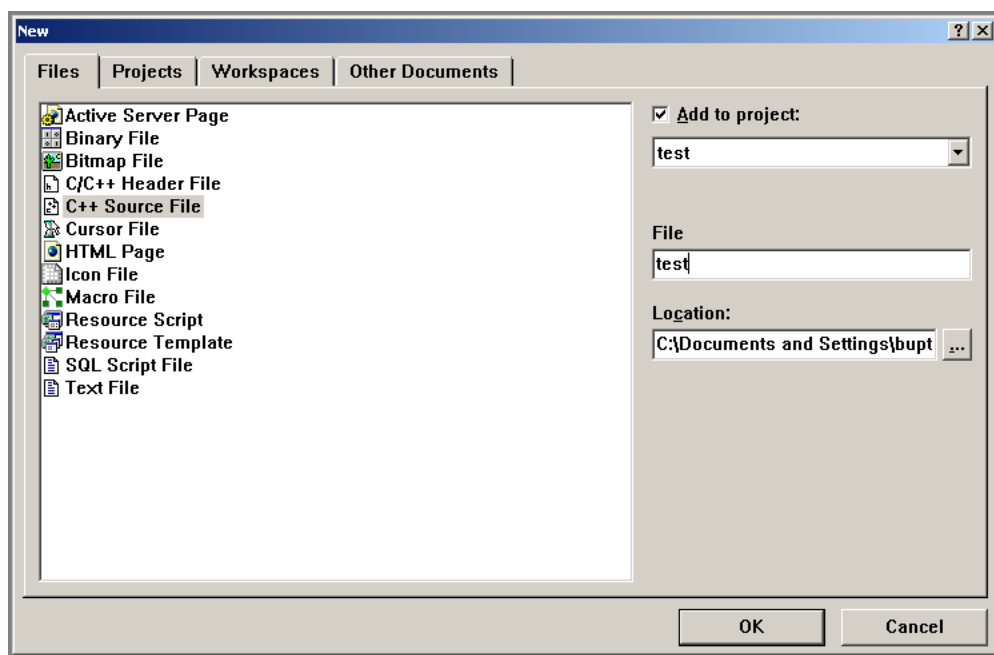
(1) 在 VC 中，点击 File->new->Projects->Win32 Console Application，右上角设置好 project 的名称和存放路径。



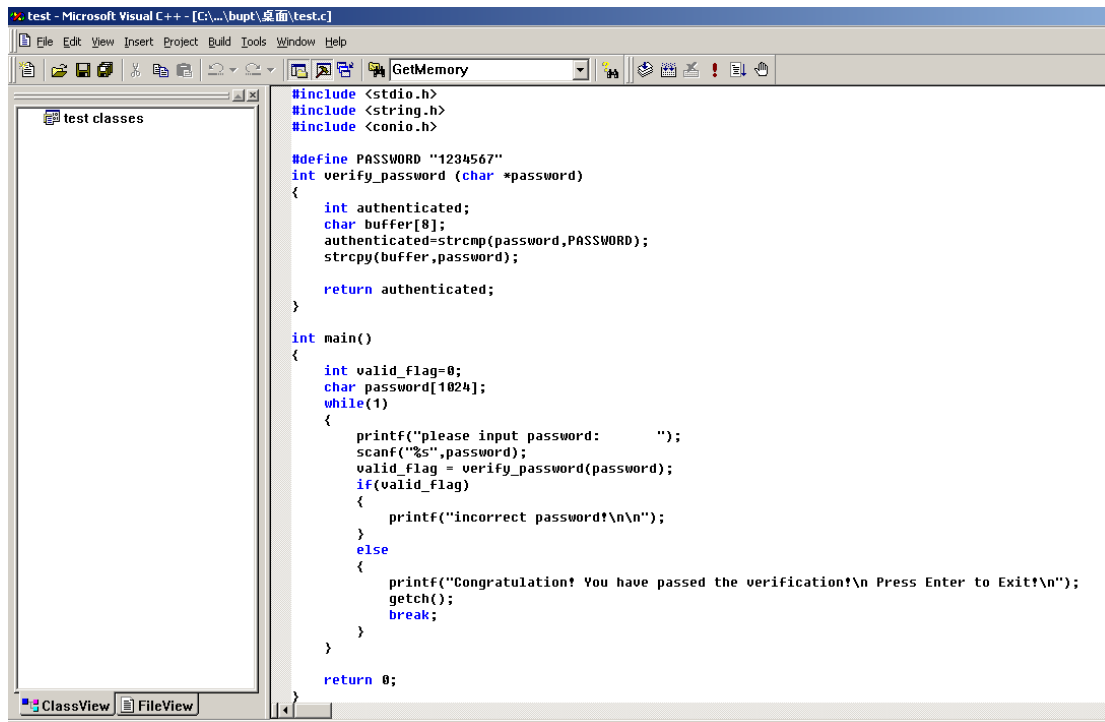
(2) 点击 OK->An empty project->Finish



(3) 之后再 new->Files->C++ Source File, 右侧设定好添加到的项目(我们刚才新建的项目), 文件名和路径。



(4) 完成新建文件后, 在新建的文件下编写或复制刚才我们看到的 main 程序, 然点击右上的红色感叹号执行。



```
#include <stdio.h>
#include <string.h>
#include <conio.h>

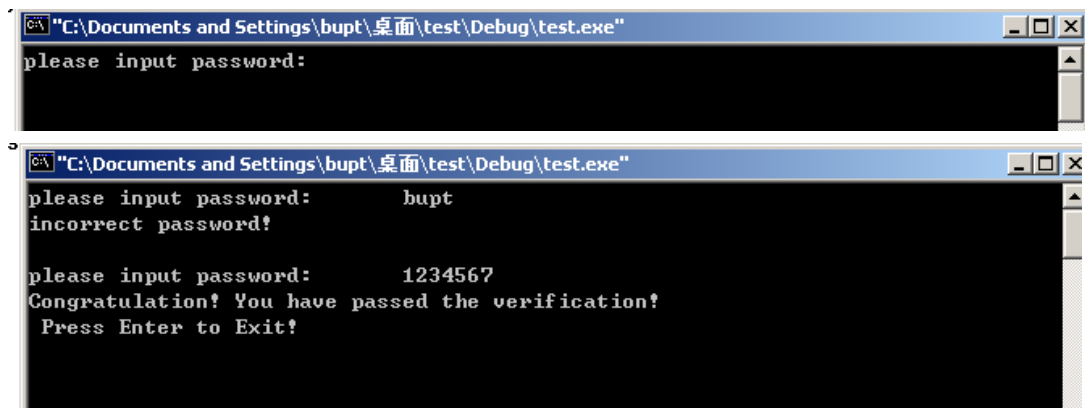
#define PASSWORD "1234567"
int verify_password(char *password)
{
    int authenticated;
    char buffer[8];
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password);

    return authenticated;
}

int main()
{
    int valid_flag=0;
    char password[1024];
    while(1)
    {
        printf("please input password: ");
        scanf("%s",password);
        valid_flag = verify_password(password);
        if(valid_flag)
        {
            printf("incorrect password!\n\n");
        }
        else
        {
            printf("Congratulation! You have passed the verification!\n Press Enter to Exit!\n");
            getch();
            break;
        }
    }

    return 0;
}
```

5.运行程序，验证以上分析。



```
C:\Documents and Settings\bupt\桌面\test\Debug\test.exe
please input password:

C:\Documents and Settings\bupt\桌面\test\Debug\test.exe
please input password:      bupt
incorrect password!

please input password:      1234567
Congratulation! You have passed the verification!
Press Enter to Exit!
```

由运行结果可知，当所输入的字符串不是 PASSWORD 的值 1234567 时，输出“incorrect password!\n\n”并开始下一次循环，直至输入正确密码 1234567 时才输出“Congratulation! You have passed the verification!\n”并跳出循环、结束程序。

（二）编写程序，改变程序的运行路线

用 python 编写密码验证小程序，程序代码如下：

```

1  user_name = "lh12004"
2  pwd = "1234567"
3
4  while True:
5      username_input = input("please input username: ")
6
7      if(username_input == user_name):
8          while True:
9              pwd_input = input("please input password: ")
10             if (pwd_input == pwd):
11                 print("Congratulation! You have passed the verification!\n")
12                 break
13             else:
14                 print("incorrect password!\n")
15             break
16         else:
17             print("incorrect username!\n")
18

```

同样使用 while True 不断循环，其中使用了两层循环，分别判断用户名和密码是否正确。外层循环为——若用户名正确，则开始内层循环验证密码；若用户名错误，则用户重新输入用户名。内层循环为——若密码正确，则验证成功跳出循环；若密码错误，则验证失败用户重新输入密码。

2.程序可以实现的功能为：当用户同时输入正确的用户名和正确密码时，才能够通过验证。

3.运行程序，测试结果如下图所示：

```

please input username: lh1
incorrect username!

please input username: lh1
incorrect username!

please input username: 123456
incorrect username!

please input username: 123456
incorrect username!

please input username: lh12004
please input password: 1234567
Congratulation! You have passed the verification!

```

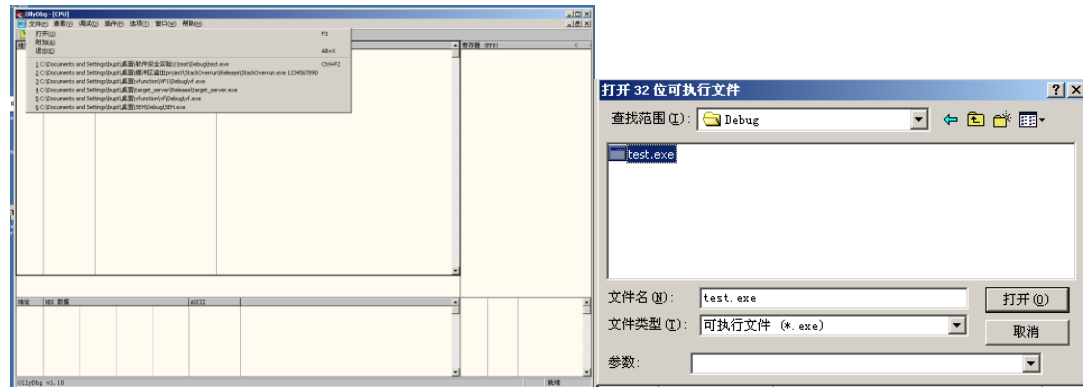
输入错误用户名时，输出“incorrect username!\n”并继续进入下一次循环；输入正确用户名时，输出“please input password: ”，用户输入密码。

输入错误密码时，输出“incorrect password!\n”并继续进入下一次循环；输入正确密码时，输出“Congratulation! You have passed the verification!\n”并跳出双层循环、结束程序。由此可知，该程序可以正确完成密码验证。

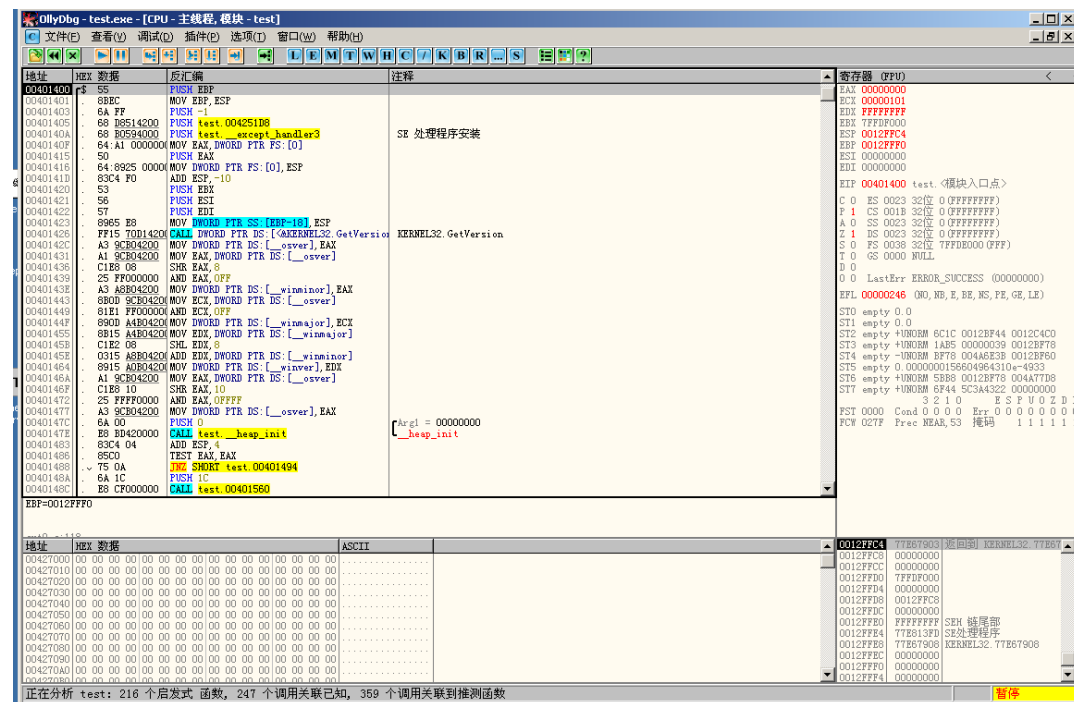
三、实验二步骤与结果

(一) 使用 Ollydbg 调试密码验证小程序，修改汇编语句以修改程序判断条件

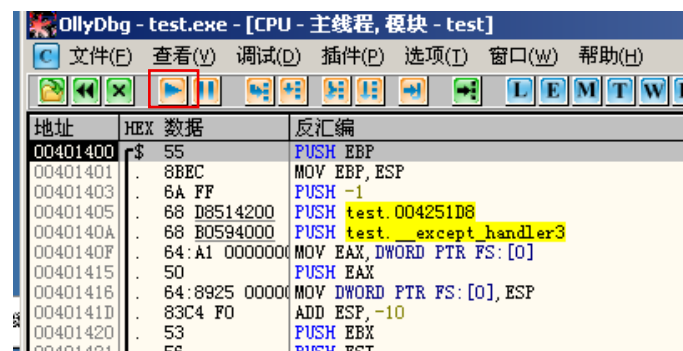
1. 运行 Ollydbg.exe，选择我们刚刚编译生成的 test.exe 文件



2. 这个 exe 文件在我们项目文件夹下的 Debug 文件夹下



3. 点击 Ollydbg 左上角运行按钮，开始运行程序。



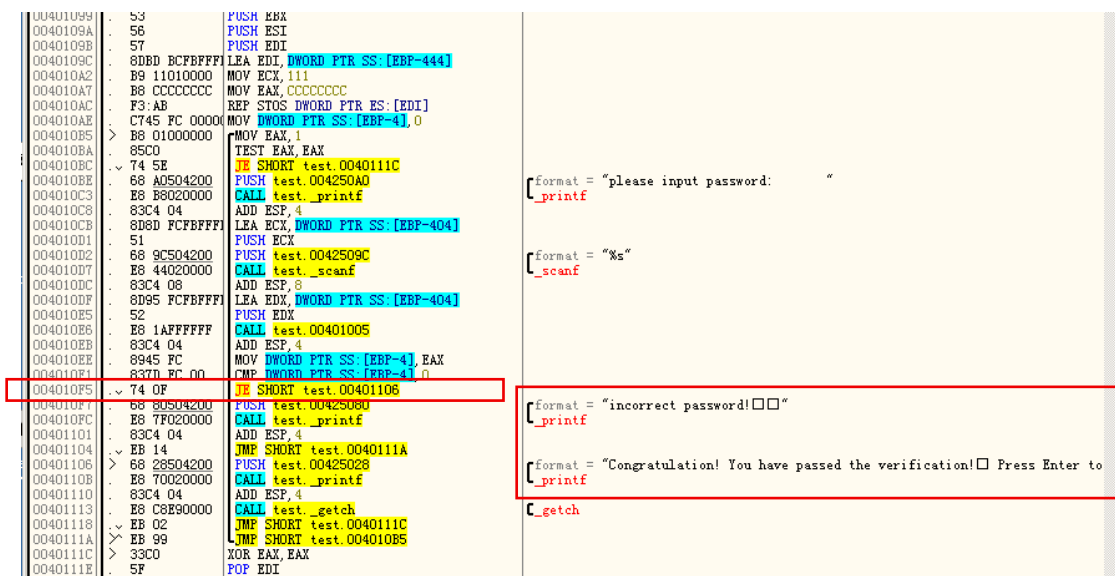
4.尝试输入几个错误密码，如“12”、“123”。

```
C:\Documents and Settings\bupt\桌面\test\Debug\test.exe
please input password:      123
incorrect password!

please input password:      12
incorrect password!

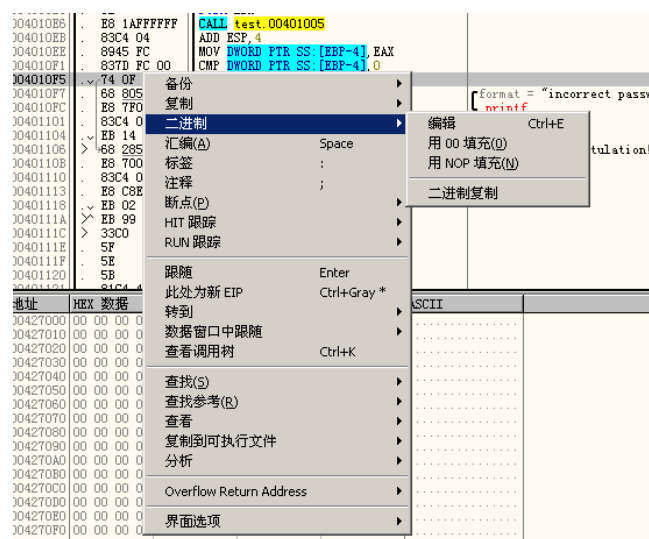
please input password:
_
```

5.可以看到 Ollydbg 的调试走到了程序判断部分，分析此部分代码：

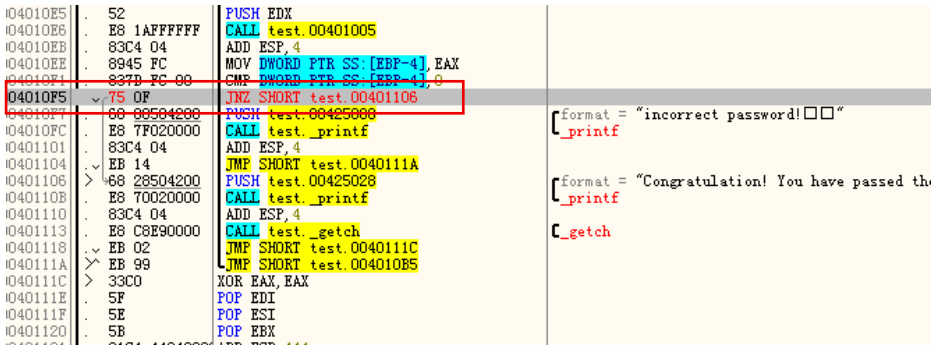
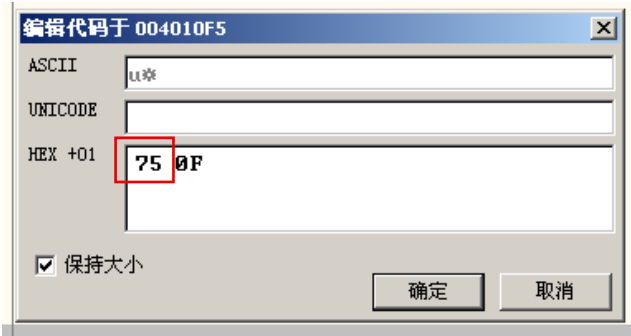


可以看到“incorrect password!”和“Congratulations! You have passed the verification!”两个字符串。

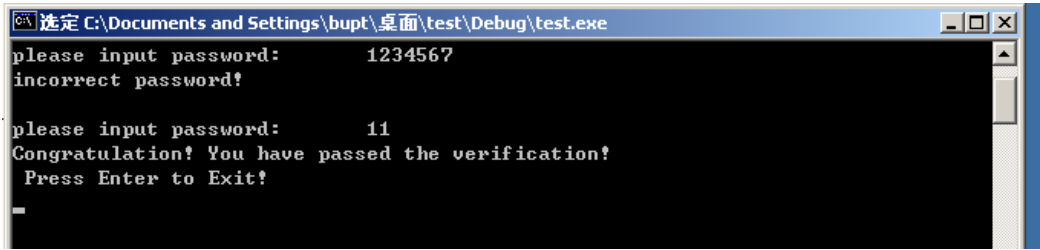
由此可知，程序 004010F5 处语句即为跳转逻辑语句，JE 即为相等时跳转。因此，为了破解此密码验证小程序，将此处的跳转条件 JE 修改为 JNE，即可在输入错误密码时通过验证。



6.修改后可以看到，此语句的操作码由 74（JE）改为了 75（JNE）。

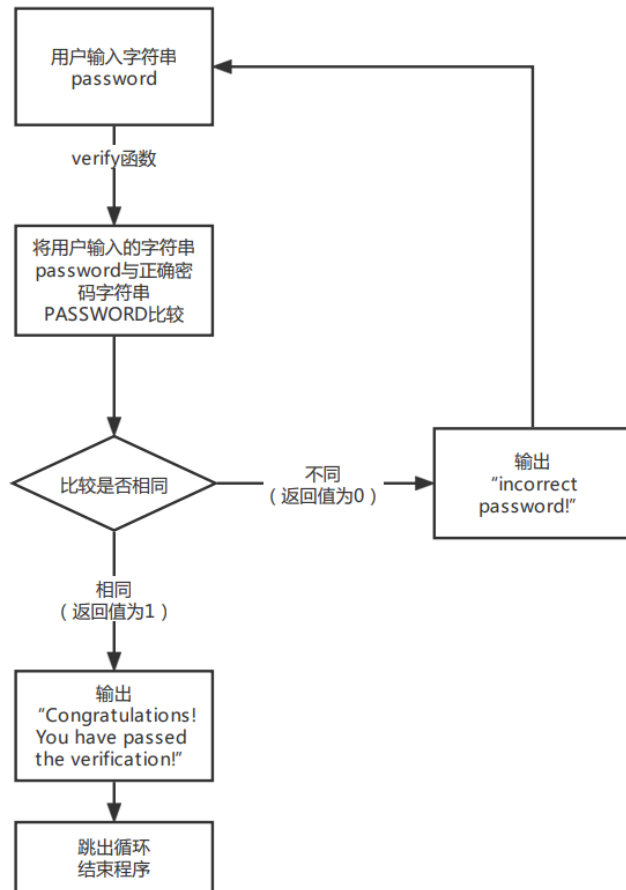


7.验证修改后的程序。修改后运行程序，输入正确密码“1234567”发现程序输出“incorrect password!”，而随意输入错误密码时则通过验证，说明已成功修改程序。

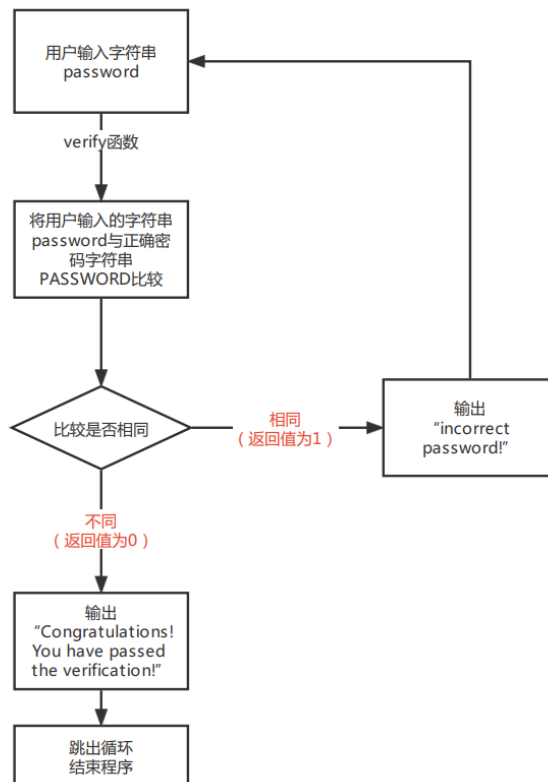


（二）程序运行结构图及修改原理图示

1.程序运行结构图：



2.修改原理图示:



（三）实验结论

简单的判断密码字符串是否相同的跳转逻辑代码对于动态调试的防御能力较弱，可以通过修改汇编语句较容易地改变判断逻辑，实现程序破解。因此我们在设计程序时，要充分考虑如何设计逻辑以抵御动态调试等破解方法，以保证软件安全。

我们只需修改一个小小的跳转条件即可实现对程序的破解，输入错误的密码提示正确，正确的密码反而提示错误。通过这次实验，我对软件安全有了更深刻的认识，也意识到软件存在着极大的安全隐患，软件安全是软件研究中一个重要方面，我们应该通过软件安全这门课程学习更多安全知识，以更好地维护网络安全。在今后的学习和工作中，编写程序时应该考虑到攻击者会进行反汇编以及修改，最好对代码进行一定的混淆。

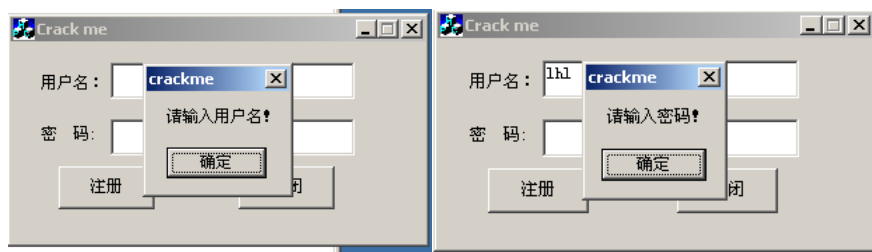
四、实验三步骤与结果

（一）运行程序初步尝试

1.首先双击 crackme.exe 运行程序，程序界面如下：



2.尝试在用户名及密码未输入的情况下点击确定，发现弹出窗口提示“请输入用户名!”“请输入密码!”。由此可知，程序中设置了判断用户名及密码是否为空的代码逻辑。

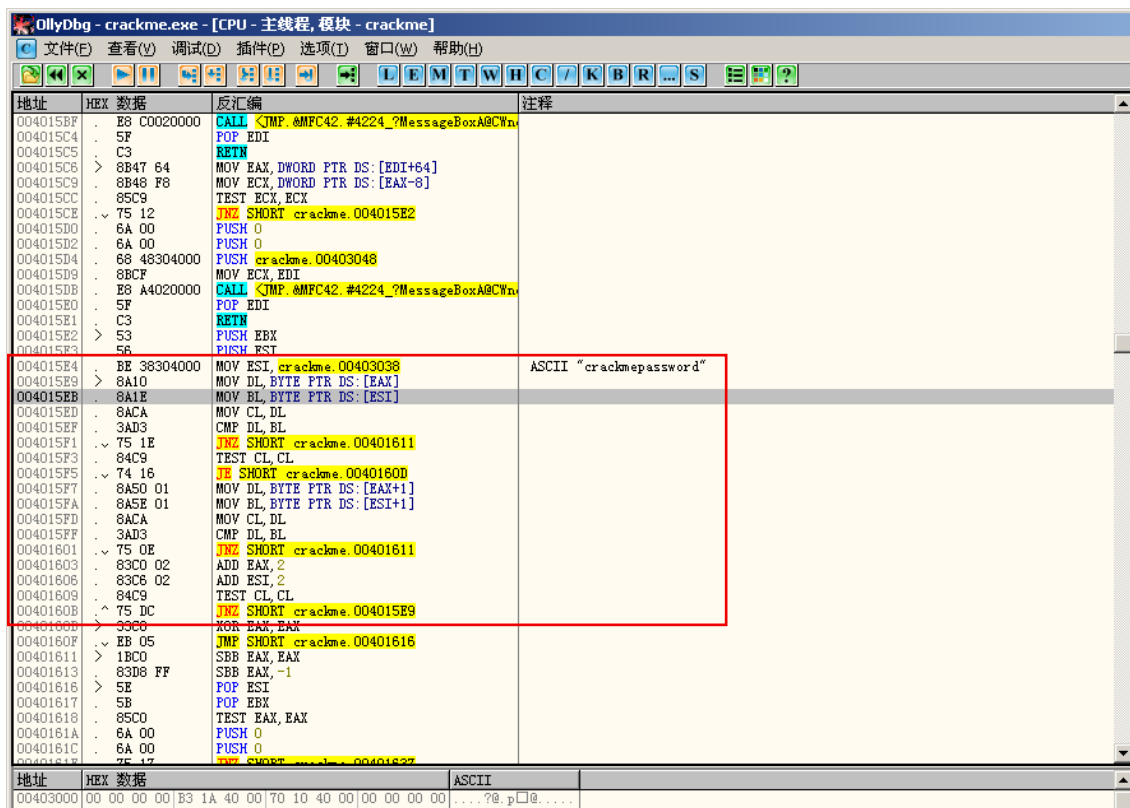


3.随意输入用户名“222”和密码“222”点击注册后，弹窗提示“注册失败!”。



(二) Ollydbg 动态调试 crackme.exe

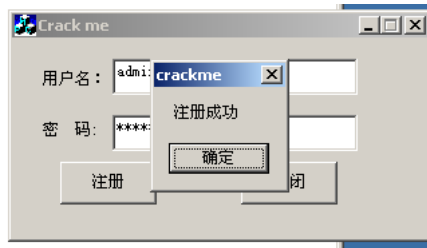
1. 在 Ollydbg 查找字符串，查找到字符串“crackmepassword”。找到后双击字符串，可以找到程序中比较判断部分逻辑代码，如下图所示。由此可知，正确密码即为字符串“crackmepassword”。



2. 重新运行程序，尝试输入密码“crackmepassword”，任意输入用户名，即可注册成功。



用户名为“222”



用户名为“admin”

3.由 Ollydbg 动态调试汇编代码可知，程序判断的关键跳转逻辑在程序的 0040161E 处，JNZ 表示比较结果不同则跳转。下方的 call 语句均为程序输出部分，负责弹窗输出“注册失败!”及“注册成功!”。

地址	HEX 数据	反汇编	注释
004015FD	. 8ACA	MOV CL, DL	
004015FF	. 3AD3	CMP DL, BL	
00401601	. 75 0E	JNZ SHORT crackme.00401611	
00401603	. 83C0 02	ADD EAX, 2	
00401606	. 83C6 02	ADD ESI, 2	
00401609	. 84C9	TEST CL, CL	
0040160B	. 75 DC	JNZ SHORT crackme.004015E9	
0040160D	. 33C0	XOR EAX, EAX	
0040160F	. EB 05	JMP SHORT crackme.00401616	
00401611	. 1BC0	SBB EAX, EAX	
00401613	. 83D8 FF	SBB EAX, -1	
00401616	. 5E	POP ESI	
00401617	. 5B	POP EBX	
00401618	. 85C0	TEST EAX, EAX	
0040161A	. 6A 00	PUSH 0	
0040161C	. 6A 00	PUSH 0	
0040161E	. 75 17	JNZ SHORT crackme.00401637	
00401620	. 68 2C304000	PUSH crackme.0040302C	
00401625	. 8BCF	MOV ECX, EDI	
00401627	. E8 58020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWn	
0040162C	. 6A 00	PUSH 0	
0040162E	. 8BCF	MOV ECX, EDI	
00401630	. E8 55020000	CALL <JMP.&MFC42.#6334_?UpdateData@CWnd	
00401635	. 5F	POP EDI	
00401636	. C3	RETN	
00401637	. 68 20304000	PUSH crackme.00403020	
0040163C	. 8BCF	MOV ECX, EDI	
0040163E	. E8 41020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWn	
00401643	. 8B07	MOV EAX, DWORD PTR DS:[EDI]	
00401645	. 8BCF	MOV ECX, EDI	
00401647	. FF90 D0000000	CALL DWORD PTR DS:[EAX+D0]	
0040164D	. 6A 00	PUSH 0	
0040164F	. 8BCF	MOV ECX, EDI	
00401651	. E8 34020000	CALL <JMP.&MFC42.#6334_?UpdateData@CWnd	
00401656	. 5F	POP EDI	
00401657	. C3	RETN	

4.因此，字符串“crackmepassword”上方语句即为弹出程序初始窗口、程序判断用户名及密码是否为空并弹窗输出“请输入用户名!”“请输入密码!”的汇编代码。

地址	HEX 数据	反汇编	注释
0040159A	90	NOP	
0040159B	90	NOP	
0040159C	90	NOP	
0040159D	90	NOP	
0040159E	90	NOP	
0040159F	90	NOP	
004015A0	57	PUSH EDI	
004015A1	8BF9	MOV EDI, ECX	
004015A3	6A 01	PUSH 1	
004015A5	E8 E0020000	CALL <JMP.&MFC42.#6334_?UpdateData@CWnd	
004015AA	8B47 60	MOV EAX, DWORD PTR DS:[EDI+60]	
004015AD	8B48 F8	MOV ECX, DWORD PTR DS:[EAX-8]	
004015B0	85C9	TEST ECX, ECX	
004015B2	75 12	JNZ SHORT crackme.004015C6	
004015B4	6A 00	PUSH 0	
004015B6	6A 00	PUSH 0	
004015B8	68 58304000	PUSH crackme.00403058	
004015BD	8BCF	MOV ECX, EDI	
004015BF	E8 C0020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWnd	
004015C4	5F	POP EDI	
004015C5	C3	RETN	
004015C6	8B47 64	MOV EAX, DWORD PTR DS:[EDI+64]	
004015C9	8B48 F8	MOV ECX, DWORD PTR DS:[EAX-8]	
004015CC	85C9	TEST ECX, ECX	
004015CE	75 12	JNZ SHORT crackme.004015E2	
004015D0	6A 00	PUSH 0	
004015D2	6A 00	PUSH 0	
004015D4	68 48304000	PUSH crackme.00403048	
004015D9	8BCF	MOV ECX, EDI	
004015DB	E8 A4020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWnd	
004015E0	5F	POP EDI	
004015E1	C3	RETN	
004015E2	53	PUSH EBX	
004015E3	56	PUSH ESI	
004015E4	BE 38304000	MOV ESI, crackme.00403038	ASCII "crackmepassword"
004015E9	8A10	MOV DL, BYTE PTR DS:[EAX]	
004015EB	8A1E	MOV BL, BYTE PTR DS:[ESI]	
004015ED	8ACA	MOV CL, DL	
004015EF	3AD3	CMP DL, BL	
004015F1	75 1E	JNZ SHORT crackme.00401611	
004015F3	84C9	TEST CL, CL	
004015F5	74 16	JZ SHORT crackme.0040160D	
004015F7	8A00 01	MOV DL, BYTE PTR DS:[EAX+1]	

(三) 三种方法破解 crackme.exe 程序

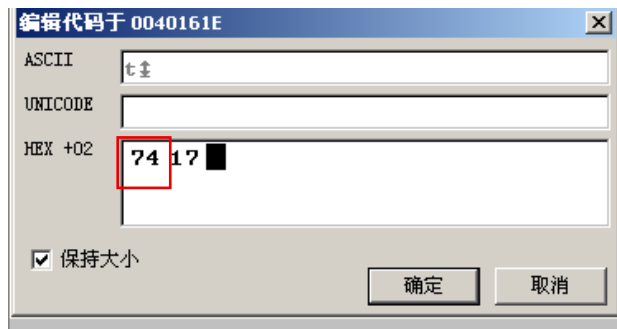
方法一：将跳转逻辑中 JNZ 修改为 JZ

将程序 0040161E 处汇编代码中不为零时跳转(JNZ)修改为零时跳转(JZ)，即将此汇编语句中操作码由 75 修改为 74，即可在输入错误密码时注册成功，而在输入正确密码时注册失败。

0040161E	6A 00	PUSH 0	
0040161F	75 17	JNZ SHORT crackme.00401637	
00401620	68 2C304000	PUSH crackme.0040302C	
00401625	8BCF	MOV ECX, EDI	
00401627	E8 58020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWnd	
0040162C	6A 00	PUSH 0	
0040162E	8BCF	MOV ECX, EDI	
00401630	E8 55020000	CALL <JMP.&MFC42.#6334_?UpdateData@CWnd	
00401635	5F	POP EDI	
00401636	C3	RETN	
00401637	68 20304000	PUSH crackme.00403020	
0040163C	8BCF	MOV ECX, EDI	
0040163E	E8 41020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWnd	
00401643	8B07	MOV EAX, DWORD PTR DS:[EDI]	
00401645	8BCF	MOV ECX, EDI	
00401647	FF90 D0000000	CALL DWORD PTR DS:[EAX+D0]	
0040164D	6A 00	PUSH 0	
0040164F	8BCF	MOV ECX, EDI	
00401651	E8 34020000	CALL <JMP.&MFC42.#6334_?UpdateData@CWnd	
00401656	5F	POP EDI	
00401657	C3	RETN	

地址	HEX 数据	ASCII
00403000	00 00 00 00 B3 1A 40 00 70 10 40 00 00 00 00 00	?@.p□@....
00403010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00403020	D7 A2 B2 E1 CA A7 B0 DC 00 00 00 00 D7 A2 B2 E1	注册失败...注册
00403030	B3 C9 B9 A6 00 00 00 00 63 72 61 63 6B 6D 65 70	成功...crackme
00403040	61 73 73 77 6F 72 64 00 C7 EB CA E4 C8 EB C3 DC	assword.请输入密
00403050	C2 EB A3 A1 00 00 00 00 C7 EB CA E4 C8 EB D3 C3	码?...请输入用
00403060	BB A7 C3 FB A3 A1 00 00 00 00 00 00 00 00 00 00	户名!.....
00403070	01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	□.....
00403080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00403090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004030A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004030B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

由图可知，JNZ SHORT crackme.00401637 即为注册成功/失败判断跳转。



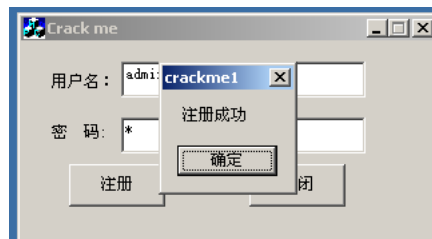
0040161A	. 6A 00	PUSH 0
0040161C	. 6A 00	PUSH 0
0040161E	74 17	JE SHORT crackme.00401637
00401620	. 68 2C304000	PUSH crackme.0040302C
00401625	. 8BCF	MOV ECX, EDI
00401627	. E8 58020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWin
0040162C	. 6A 00	PUSH 0
0040162E	. 8BCF	MOV ECX, EDI

修改：将 JNZ SHORT crackme.00401637 改成 JE SHORT crackme.00401637。

修改后再次运行程序，如下两图可知，输入任意错误密码可以注册成功，成功实现破解 crackme.exe 程序。



用户名为：“admin”，密码为“crackmepassword”时注册失败



用户名为“admin”，密码为“1”时注册成功

方法二：将密码（或用户名）为空的跳转逻辑修改为跳转至注册成功

1.由（二）中对汇编代码的分析可知，程序 004015CE 处的代码即为程序判断密码为空时实现跳转到“请输入密码”弹窗。为了验证这个结论，在程序调用 MessageBox 处（004015DB）双击设置断点后运行程序。

CPU - 主线程, 模块 - crackme				
地址	HEX	数据	反汇编	注释
004015B7	.	E8 C0020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWin	
004015C4	.	5F	POP EDI	
004015C5	.	C3	RETN	
004015C8	>	8E47 64	MOV EAX, DWORD PTR DS:[EDI+64]	
004015C9	.	8E48 F8	MOV ECX, DWORD PTR DS:[EAX-8]	
004015CC	.	85C9	TEST ECX, ECX	
004015CE	√	75 12	JNZ SHORT crackme.004015E2	
004015D0	.	6A 00	PUSH 0	
004015D2	.	6A 00	PUSH 0	
004015D4	.	68 48304000	PUSH crackme.00403048	
004015D9	.	8BCF	MOV ECX, EDI	
004015DB	.	E8 A4020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWin	
004015E0	.	5F	POP EDI	
004015E1	.	C3	RETN	
004015E2	>	53	PUSH EBX	
004015E3	.	56	PUSH ESI	
004015E4	.	BE 38304000	MOV ESI, crackme.00403038	
004015E9	>	8A10	MOV DL, BYTE PTR DS:[EAX]	ASCII "crackmepassword"
004015EB	.	8A1E	MOV BL, BYTE PTR DS:[ESI]	
004015ED	.	8ACA	MOV CL, DL	
004015EF	.	3AD3	CMP DL, BL	
004015F1	√	75 1E	JNZ SHORT crackme.00401611	
004015F3	.	84C9	TEST CL, CL	
004015F5	√	74 16	JE SHORT crackme.0040160D	
004015F7	.	8A50 01	MOV DL, BYTE PTR DS:[EAX+1]	
004015FA	.	8A5E 01	MOV BL, BYTE PTR DS:[ESI+1]	
004015FD	.	8ACA	MOV CL, DL	
004015FF	.	3AD3	CMP DL, BL	
00401601	√	75 0E	JNZ SHORT crackme.00401611	
00401603	.	83C0 02	ADD EAX, 2	
00401606	.	83C6 02	ADD ESI, 2	
00401609	.	84C9	TEST CL, CL	

在 004015DB 处设置断点

```

004015CC . 85C9      TEST ECX, ECX
004015CE . 75 12     JNZ SHORT crackme.004015E2
004015D0 . 6A 00     PUSH 0
004015D2 . 6A 00     PUSH 0
004015D4 . 68 48304000 PUSH crackme.00403048
004015D9 . 8BCF     MOV ECX, EDI
004015DB . E8 A4020000 CALL <JMP.&MFC42.#4224_?MessageBoxA@CWin
004015E0 . 5F       POP EDI
004015E1 . C3       RETN
004015E2 > 53       PUSH EBX
004015E3 . 56       PUSH ESI
004015E4 . BE 38304000 MOV ESI, crackme.00403038
004015E9 > 8A10     MOV DL, BYTE PTR DS:[EAX]
004015EB . 8A1E     MOV BL, BYTE PTR DS:[ESI]
004015ED . 8ACA     MOV CL, DL
004015EF . 3AD3     CMP DL, BL
004015F1 . 75 1E     JNZ SHORT crackme.00401611
004015F3 . 84C9     TEST CL, CL
004015F5 . 74 16     JE SHORT crackme.0040160D
004015F7 . 8A50 01  MOV DL, BYTE PTR DS:[EAX+1]
004015FA . 8A5E 01  MOV BL, BYTE PTR DS:[ESI+1]
004015FD . 8ACA     MOV CL, DL
004015FF . 3AD3     CMP DL, BL
00401601 . 75 0E     JNZ SHORT crackme.00401611
00401603 . 83C0 02  ADD EAX, 2
00401606 . 83C6 02  ADD ESI, 2
00401609 . 84C9     TEST CL, CL

```

输入用户名为“admin”，密码为空

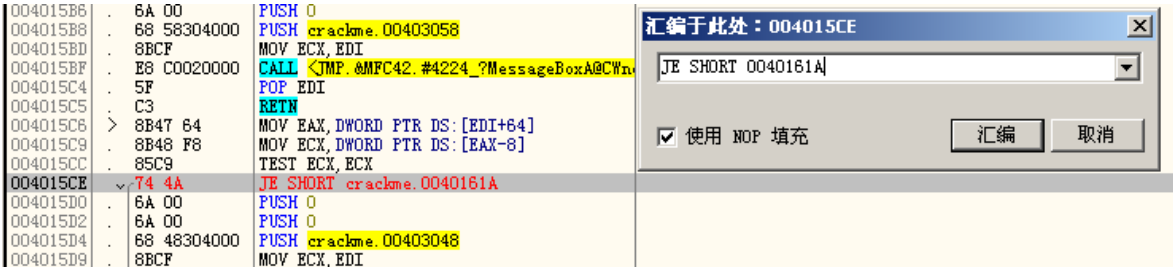
004015CC	.	85C9	TEST ECX, ECX	
004015CE	√	75 12	JNZ SHORT crackme.004015E2	
004015D0	.	6A 00	PUSH 0	
004015D2	.	6A 00	PUSH 0	
004015D4	.	68 48304000	PUSH crackme.00403048	
004015D9	.	8BCF	MOV ECX, EDI	
004015DB	.	E8 A4020000	CALL <JMP.&MFC42.#4224_?MessageBoxA@CWin	
004015E0	.	5F	POP EDI	
004015E1	.	C3	RETN	
004015E2	>	53	PUSH EBX	
004015E3	.	56	PUSH ESI	
004015E4	.	BE 38304000	MOV ESI, crackme.00403038	
004015E9	>	8A10	MOV DL, BYTE PTR DS:[EAX]	ASCII "crackmepassword"
004015EB	.	8A1E	MOV BL, BYTE PTR DS:[ESI]	
004015ED	.	8ACA	MOV CL, DL	
004015EF	.	3AD3	CMP DL, BL	
004015F1	√	75 1E	JNZ SHORT crackme.00401611	
004015F3	.	84C9	TEST CL, CL	
004015F5	√	74 16	JE SHORT crackme.0040160D	

点击注册后，程序运行到断点处

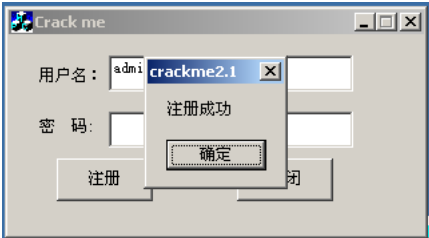
2.由以上结果可知，004015CE 处为程序判断密码是否为空的跳转逻辑代码。为了修改相应弹窗内容，要使这部分入栈及调用 MessageBox 代码全部替换为“注

册成功”弹窗的入栈及调用 MessageBox 代码。

因此，将程序 004015CE 跳转逻辑由“当不为零（密码不为空）时跳转至 0015E2 进行密码字符串的比较”修改为“当为零（密码为空）时跳转至 0040161A 处进行注册成功弹窗”。即将汇编语句修改为 JZ SHORT 0040161A。



3.修改后运行程序，输入用户名为 admin，密码为空时即可显示“注册成功！”弹窗。



输入用户名为“admin”，密码为空时显示注册成功

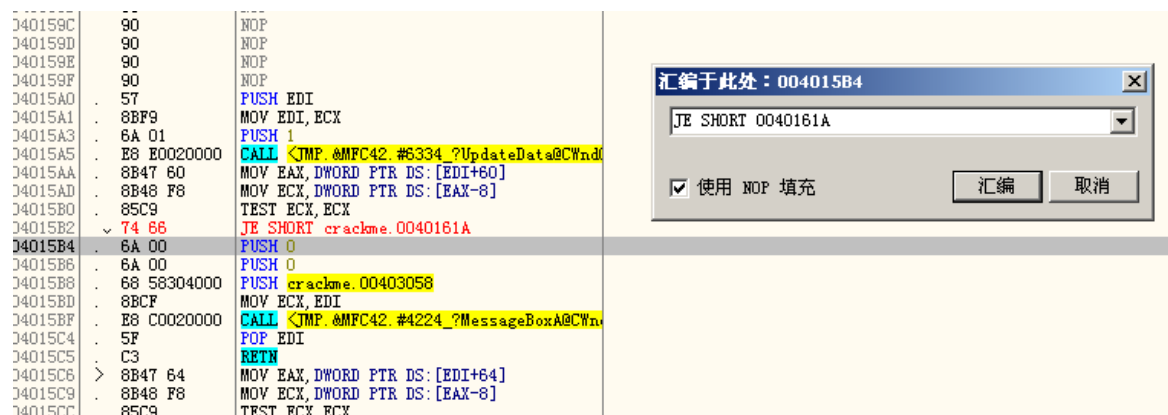


输入用户名为“admin”，密码为正确密码“crackmepassword”显示请输入密码



输入用户名为“admin”，密码为错误密码“1”显示请输入密码

4.同理可知，也可以将程序跳转逻辑修改为“当用户名为空时，跳转到显示注册成功弹窗的代码”。即将程序 004015B2 处汇编语句修改为 JZ SHORT 0040161A。修改完成后运行程序，用户名为空时不进行输入直接点击注册，显示弹窗“注册成功”。



用户名为空时显示注册成功

至此实现破解 crackme.exe 程序。

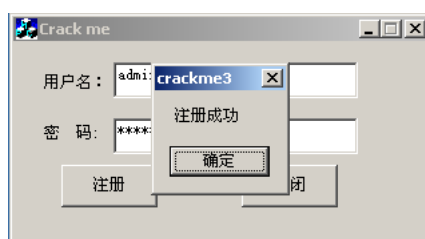
方法三：将正确密码字符串修改为“mycrackpassword”

1.找到密码字符串“crackmepassword”，在下方 HEX 数据框中根据 ASCII 码将密码字符串修改为“mycrackpassword”。

CPU - 主线程, 模块 - crackme				
地址	HEX 数据	反汇编	注释	
004015E0	5F	POP EDI		
004015E1	C3	RETN		
004015E2	53	PUSH EBX		
004015E3	56	PUSH ESI		
004015E4	BE 38304000	MOV ESI, crackme.00403038	ASCII "crackmepassword"	
004015E9	8A10	MOV DL, BYTE PTR DS:[EAX]		
004015EB	8A1E	MOV BL, BYTE PTR DS:[ESI]		
004015ED	8ACA	MOV CL, DL		
004015EF	3AD3	CMP DL, BL		
004015F1	75 1E	JNZ SHORT crackme.00401611		
004015F3	84C9	TEST CL, CL		
004015F5	74 16	JE SHORT crackme.0040160D		
004015F7	8A50 01	MOV DL, BYTE PTR DS:[EAX+1]		
004015FA	8A5E 01	MOV BL, BYTE PTR DS:[ESI+1]		
004015FD	8ACA	MOV CL, DL		
004015FF	3AD3	CMP DL, BL		
00401601	75 0E	JNZ SHORT crackme.00401611		
00401603	83C0 02	ADD EAX, 2		
00401606	83C6 02	ADD ESI, 2		
00401609	84C9	TEST CL, CL		
0040160B	75 DC	JNZ SHORT crackme.004015E9		
0040160D	33C0	XOR EAX, EAX		
0040160F	EB 05	JMP SHORT crackme.00401616		
00401611	1BC0	SBB EAX, EAX		
00401613	83D8 FF	SBB EAX, -1		
00401616	5E	POP ESI		
00401617	5B	POP EBX		
00401618	85C0	TEST EAX, EAX		
0040161A	6A 00	PUSH 0		
0040161C	6A 00	PUSH 0		
0040161E	75 17	JNZ SHORT crackme.00401637		
00401620	68 2C304000	PUSH crackme.0040302C		
地址	HEX 数据	反汇编	ASCII	
00403000	00 00 00 00 B3 1A 40 00 70 10 40 00 00 00 00 00		?@.p@.....	
00403010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00403020	D7 A2 E2 E1 CA A7 B0 DC 00 00 00 00 D7 A2 E2 E1		注册失败...注册	
00403030	B3 C9 B9 A5 00 00 00 00 63 72 61 63 6E 6D 65 70		成功...crackme	
00403040	61 73 73 77 6F 72 64 00 C7 EB CA E4 C8 EB C3 DC		assword.请输入密	
00403050	C2 EB A3 A1 00 00 00 00 C7 EB E4 C8 EB D3 C3		码? ...请输入用	
00403060	EB A7 C3 FB A3 A1 00 00 00 00 00 00 00 00 00 00		用户名?	
00403070	01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		□.....	
00403080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			
00403090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00			

地址	HEX 数据	ASCII
00403000	00 00 00 00 B3 1A 40 00 70 10 40 00 00 00 00 00	?@.p□@.
00403010	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00403020	D7 A2 B2 E1 CA A7 B0 DC 00 00 00 00 D7 A2 B2 E1	注册失败...注册
00403030	B3 C9 B9 A6 00 00 00 00 6D 79 63 72 61 63 6B 70	成功...mycrackp
00403040	61 73 73 77 6F 72 64 00 C7 EB CA E4 C8 EB C3 DC	assword.请输入密
00403050	C2 EB A3 A1 00 00 00 00 C7 EB CA E4 C8 EB D3 C3	码!...请输入用
00403060	BB A7 C3 FB A3 A1 00 00 00 00 00 00 00 00 00	户名!.....
00403070	01 00 00 00 00 00 00 00 00 00 00 00 00 00 00	□.....
00403080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00403090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

2.修改完成后点击运行程序，任意输入用户名，输入密码“mycrackpassword”，注册成功；输入原始密码“mycrackpassword”时，注册失败。成功实现破解crackme.exe 程序。



用户名为“admin”，密码为“mycrackpassword”时注册成功



用户名为“admin”，密码为“crackmepassword”时注册失败

附件：crackme 破解程序

crackme1.exe：输入任意错误密码时显示注册成功，输入正确密码crackmepassword时则显示注册失败。

crackme2.1.exe: 输入任意用户名，密码为空时显示注册成功；输入任意密码则显示“请输入密码！”

crackme2.2.exe: 用户名为空时，点击注册即显示注册成功。

crackme3.exe: 输入任意用户名，密码为修改后密码 mycrackpassword 时显示注册成功；输入正确密码或其他错误密码时则注册失败。

（四）实验结论

通过本次实验，我对软件安全中常见的身份验证绕过机制有了更加深入和直

观的认识。`crackme.exe` 作为一个典型的口令验证程序，其验证逻辑在未加保护的情况下暴露出多处可被篡改的脆弱点。实验中我采用的三种破解方法：修改关键跳转条件、利用空输入绕过验证、篡改密码明文数据。分别从控制流、输入校验和数据完整性三个层面揭示了软件在设计上的安全风险。

首先，验证逻辑中的条件跳转（JNZ/JZ）直接决定了程序的分支走向，而这一机制若暴露于可修改的内存环境中，极易被攻击者利用，从而完全颠覆原有的安全判断。其次，对用户输入的有效性检查本应是防御的第一道防线，但若检查逻辑本身存在缺陷或被篡改，反而可能成为绕过验证的入口。最后，程序中硬编码的敏感信息（如密码字符串）以明文形式存储，缺乏必要的混淆或加密措施，使得攻击者可通过静态或动态分析直接定位并修改关键数据。

这次实验也让我进一步认识到，软件安全并非仅在开发完成后通过外部防护手段来保障，而是需要在编码、编译、乃至分发前的各个环节中，系统性地融入安全思维。作为《软件安全》课程的一次实践，本实验不仅锻炼了我使用调试工具进行动态分析的能力，更深化了我对“安全在于细节”这一理念的理解——任何微小的逻辑疏漏或实现上的不慎，都可能在恶意分析面前被放大为严重的安全漏洞。今后在参与软件开发时，我将更加注重代码的健壮性、敏感信息的保护机制以及整体架构的安全性设计，从而在根源上降低此类风险的发生概率。